

Software Design Description

6.1 Introduction

This document presents the architecture and software design for the Sammy's Games project. The system is a browser-based application that implements a variety of original games, beginning with *Knockout*, a two-player board game featuring unique push mechanics and an AI-controlled opponent ("Knockout Bot").

The purpose of this document is to describe the overall architectural structure of the system, identify its major components, and explain how those components interact. This SDD provides sufficient architectural detail to guide implementation and future expansion of additional games in the platform.

6.1.1 System Objectives

The primary objectives of this project are:

- To design and implement 2–3 fully functional browser-based strategy games within one semester.
- To ensure complete and accurate rule enforcement for each game.
- To develop an AI opponent capable of making legal and intelligent decisions.
- To provide a clean and intuitive graphical user interface.

The current implementation focuses on completing and refining the *Knockout* game as the first fully operational module in the collection.

6.1.2 Hardware, Software, and Human Interfaces

6.1.2.1 Hardware Interface

The application operates on standard personal computing hardware and requires:

- A computer with a display monitor
- A mouse or trackpad for input
- A keyboard (optional, though not currently required for gameplay)
- Internet access only if hosted remotely

No specialized hardware is required. All interactions are handled through browser-based input and output.

6.1.2.2 Software Interface

The system is implemented using:

- HTML for structural layout
- CSS for styling and board layout
- JavaScript for game logic and state management
- Modern web browser (Chrome, Firefox, Safari, or Edge)

The application runs entirely client-side. No external libraries, frameworks, APIs, or server-side components are used. All logic executes locally in the browser environment.

6.1.2.3 Human Interface (Graphical User Interface)

The user interface consists of:

- A 4x4 game board rendered as a CSS Grid
- A rules display panel explaining gameplay mechanics
- A navigation bar for switching between games
- A move information panel displaying last move direction for each player
- A game-over announcement banner

User interactions are click-based and follow this general flow:

1. The user selects one of their pieces.
2. Valid moves are visually highlighted.
3. The user selects a destination square.
4. The system validates and executes the move.
5. The AI opponent responds automatically after a brief delay.

Visual feedback includes:

- Highlighted selected pieces
- Highlighted valid move squares
- Clear win notification banner

This interface design emphasizes clarity, simplicity, and responsiveness.

6.2 Architectural Design

The system follows a modular application architecture organized around logical subsystems. Although implemented in JavaScript functions, the architecture conceptually separates responsibilities to reduce coupling and improve maintainability.

The architecture is divided into the following major subsystems:

- State Management Subsystem

- Game Logic Subsystem
- AI Decision Subsystem
- User Interface Rendering Subsystem
- Event Control Subsystem

Each subsystem is responsible for a distinct aspect of the application's behavior.

6.2.1 Major Software Components

1. State Management Subsystem

This component maintains all persistent game data during execution, including:

- Board state representation (4x4 grid stored in a one-dimensional array)
- Current player turn tracking
- Last move direction tracking for each player
- Game-over state flag

It serves as the central source of truth for the application.

2. Game Logic Subsystem

This component enforces the rules of *Knockout*. Its responsibilities include:

- Validating legal movement
- Checking for obstructions during normal moves
- Enforcing push-move constraints
- Handling multi-piece push chains
- Detecting when a piece is pushed off the board
- Determining win condition

This subsystem ensures that all gameplay remains consistent with defined rules.

3. AI Decision Subsystem (Bot)

This subsystem implements the computer-controlled opponent. It:

- Generates all possible legal moves
- Simulates moves using cloned board states
- Evaluates outcomes for win detection
- Avoids moves that eliminate its own piece
- Selects an optimal or safe move

The AI currently performs single-move lookahead using simulation-based evaluation.

4. User Interface Rendering Subsystem

This subsystem is responsible for:

- Updating the visual board state
- Applying CSS classes for selected and valid squares
- Updating move history display
- Displaying win announcements

It translates internal state data into visual representation.

5. Event Control Subsystem

This subsystem coordinates user interaction and game flow. It:

- Handles click events
- Determines whether a piece is being selected or moved
- Calls validation logic
- Updates state after a move
- Triggers AI response
- Prevents interaction after game completion

This component acts as the controller coordinating other subsystems.

6.2.2 Major Software Interactions

The system operates through synchronous function calls between subsystems. There are no asynchronous network interactions.

Typical move sequence:

1. User clicks a square.
2. Event Control checks current selection state.
3. Event Control calls Game Logic to validate move.
4. If valid:
 - State Management updates board state.
 - Game Logic processes push chain (if applicable).
 - User Interface Rendering updates visual board.
5. If turn ends:
 - AI Subsystem generates and simulates moves.
 - Selected move is executed through same Game Logic.
 - UI updates accordingly.

All components share controlled access to the centralized board state array.

No external services or APIs are involved in system execution.

6.2.3 Architectural Design Diagrams

The following UML diagrams will accompany this document:

1. Use Case Diagram

Actors:

- Human Player
- AI Bot

Use Cases:

- Select Piece
- Validate Move
- Execute Move
- Execute Push Chain
- Detect Win
- Display Game Over

2. Top-Level Component/Class Diagram

Displays the relationships among:

- Game Controller (Event Control)
- Board State Manager
- Move Validator
- Push Engine
- Bot AI
- Renderer

Shows dependencies and method-call relationships.

3. Sequence Diagram – Turn Execution

Illustrates:

Human Player
→ Event Controller
→ Move Validator
→ State Manager
→ Renderer
→ Bot AI

- Move Validator
- Renderer

This diagram shows the runtime interaction flow for one full human + bot turn cycle.